

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель _____ С.В. Умбетов

« _____ » _____ 2026 г.

Отчет по лабораторной работе №2
по дисциплине «Инженерно-техническая поддержка
сопровождения интеллектуальных информационных систем»
«СОСТАВЛЕНИЕ ТЕХНИЧЕСКОГО ЗАДАНИЯ ДЛЯ ОБСЛУЖИВАНИЯ И
РЕЗЕРВНОГО КОПИРОВАНИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ»

ЛР 09.02.08.21.002

Студент группы _____ 1ИИС-41 _____ Т.Е.Рябич
группа фи.о., фамилия

Преподаватель _____ старший преподаватель, доцент _____ С.В. Умбетов
должность, ученая степень фи.о., фамилия

Барнаул 2026

Цель работы:

Получить практические навыки настройки распределённой системы контроля версий Git в операционной системе Linux, освоить основные методы управления состояниями файлов локального репозитория, а также изучить базовые принципы ветвления и бесконфликтного слияния данных.

Задачи работы:

- Проверить версию предустановленного в операционной системе пакета Git и определить характеристики текущего Linux-дистрибутива.
- Выполнить глобальную конфигурацию параметров создателя коммитов путем привязки имени автора и адреса электронной почты.
- Инициализировать локальный репозиторий в пустом рабочем каталоге проекта.
- Освоить управление жизненным циклом файлов, используя команды отслеживания, индексации, фиксирования снимков состояний и анализа построчных различий.
- Создать файл исключений и настроить в нем регулярные шаблоны для автоматического игнорирования нежелательных системных объектов.
- Изучить механизмы ветвления структуры проекта путем создания изолированной ветки, внесения правок, переключения между указателями и выполнения слияния данных.
- Проанализировать полную итоговую историю коммитов репозитория с помощью графической визуализации.

Задание к работе: Выполнить 20 шагов по инициализации локального репозитория в РЕД ОС. Настроить данные автора, создать тестовые файлы и сконфигурировать .gitignore. Отработать создание ветки feature/update-readme, внесение изменений в файл readme.txt и последующее слияние с веткой master по схеме *Fast-forward*.

Задание принял: _____



Подпись

Рябич Т.Е.

Ход работы

ЗАДАНИЕ 1. ПРОВЕРИТЬ НАЛИЧИЕ GIT В СИСТЕМЕ

Открыл консольный терминал и выполнил специальную команду для проверки наличия распределённой системы контроля версий Git в операционной системе. Консоль успешно вернула строку с указанием текущей установленной версии ПО, что подтвердило присутствие Git в системе и отсутствие необходимости его повторной установки.

```
[24050088@350-glк-рс-09 ~]$ git --version  
git version 2.53.0
```

Рисунок 1 – Проверка версии установленного Git

ЗАДАНИЕ 2. ОПРЕДЕЛИТЬ ДИСТРИБУТИВ LINUX

Для идентификации параметров окружения операционной системы и корректного определения характеристик рабочей среды считал содержимое системного файла релиза с помощью команды `cat /etc/os-release`. В выводе терминала зафиксировал параметры отечественного дистрибутива **RED OS 8.0.3 (Education Desktop)** на базе пакетной базы `rhel centos fedora`.

```
[24050088@350-glк-рс-09 ~]$ cat /etc/os-release  
NAME="RED OS"  
VERSION="8.0.3"  
PLATFORM_ID="platform:red80"  
ID="redos"  
ID_LIKE="rhel centos fedora"  
VERSION_ID="8.0.3"  
PRETTY_NAME="RED OS 8.0.3 (Education Desktop)"  
ANSI_COLOR="0;31"  
CPE_NAME="cpe:/o:redos:redos:8"  
HOME_URL="https://redos.red-soft.ru"  
BUG_REPORT_URL="https://support.red-soft.ru"  
EDITION="Education"
```

Рисунок 2 – Информация о дистрибутиве RED OS

ЗАДАНИЕ 3. НАСТРОИТЬ ИМЯ ПОЛЬЗОВАТЕЛЯ

Для автоматического заполнения метаданных всех будущих коммитов и однозначной идентификации автора изменений задал глобальное имя пользователя **ryabich_1iis_41** в конфигурации Git.

```
[24050088@350-glк-рс-09 ~]$ git config --global user.name "ryabich_1iis_41"  
[24050088@350-glк-рс-09 ~]$
```

Рисунок 3 – Настройка глобального имени пользователя

ЗАДАНИЕ 4. НАСТРОИТЬ АДРЕС ЭЛЕКТРОННОЙ ПОЧТЫ

Аналогичным образом через терминал прописал глобальный email пользователя your.email@example.com, который будет автоматически привязываться к метаданным каждой создаваемой контрольной точки.

```
[24050088@350-glk-pc-09 ~]$ git config --global user.email your.email@example.com  
[24050088@350-glk-pc-09 ~]$
```

Рисунок 4 – Настройка глобального email пользователя

ЗАДАНИЕ 5. ПРОВЕРИТЬ ПРИМЕНЁННЫЕ НАСТРОЙКИ

Для контроля корректности внесения изменений в глобальный конфигурационный файл вывел полный список текущих настроек Git с помощью команды `git config -list` и убедился, что параметры отображают верные значения.

```
[24050088@350-glk-pc-09 ~]$ git config --list  
user.name=ryabich_liis_41  
user.email=your.email@example.com  
[24050088@350-glk-pc-09 ~]$
```

Рисунок 5 – Просмотр списка глобальных настроек Git

ЗАДАНИЕ 6. СОЗДАТЬ РАБОЧУЮ ДИРЕКТОРИЮ

С помощью стандартных команд командной строки Linux создал новую папку для практической работы под названием `git-practice` и осуществил непосредственный переход в неё через терминал.

```
[24050088@350-glk-pc-09 ~]$ mkdir git-practice  
[24050088@350-glk-pc-09 ~]$ cd git-practice  
[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 6 – Создание и переход в рабочую директорию

ЗАДАНИЕ 7. ИНИЦИАЛИЗИРОВАТЬ ЛОКАЛЬНЫЙ РЕПОЗИТОРИЙ

Выполнил команду инициализации репозитория в текущем каталоге. Система сгенерировала пустой локальный репозиторий Git и вывела уведомление об использовании ветки по умолчанию (`master`).

```
[24050088@350-glk-pc-09 git-practice]$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Инициализирован пустой репозиторий Git в /home/24050088@ad.astu/git-practice/.git/
[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 7 – Инициализация нового Git-репозитория

ЗАДАНИЕ 8. УБЕДИТЬСЯ В СОЗДАНИИ СКРЫТОЙ ПАПКИ .GIT

Проверил физическое создание скрытой системной директории, в которой Git хранит всю базу данных, конфигурационные файлы и историю изменений проекта. В списке отобразился скрытый каталог .git.

```
[24050088@350-glk-pc-09 git-practice]$ ls -la
итого 12
drwxr-xr-x. 3 24050088 пользователи домена 4096 сен 22 15:51 .
drwx----- 17 24050088 пользователи домена 4096 сен 22 15:49 ..
drwxr-xr-x. 6 24050088 пользователи домена 4096 сен 22 15:51 .git
[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 8 – Проверка наличия папки .git через ls -la

ЗАДАНИЕ 9. СОЗДАТЬ ТЕСТОВЫЕ ФАЙЛЫ

С помощью утилиты touch сгенерировал три пустых файла различных форматов (readme.txt, script.sh, data.json) для последующего контроля их состояний в репозитории.

```
[24050088@350-glk-pc-09 git-practice]$ touch readme.txt
[24050088@350-glk-pc-09 git-practice]$ touch script.sh
[24050088@350-glk-pc-09 git-practice]$ touch data.json
[24050088@350-glk-pc-09 git-practice]$ ls
data.json  readme.txt  script.sh
```

Рисунок 9 – Создание файлов readme.txt, script.sh и data.json

ЗАДАНИЕ 10. ПРОВЕРИТЬ СТАТУС РЕПОЗИТОРИЯ

Вывел текущий статус локального репозитория. Git зафиксировал, что все три созданных файла находятся в состоянии *untracked* (неотслеживаемые), поскольку они еще не были добавлены в индекс.

```
[24050088@350-glk-pc-09 git-practice]$ git status
Текущая ветка: master

Еще нет коммитов

Неотслеживаемые файлы:
(используйте «git add <файл>...», чтобы добавить в то, что будет включено в коммит)
    data.json
    readme.txt
    script.sh

индекс пуст, но есть неотслеживаемые файлы
(используйте «git add», чтобы проиндексировать их)
[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 10 – Статус репозитория — файлы в состоянии *untracked*

ЗАДАНИЕ 11. ДОБАВИТЬ В ИНДЕКС ТОЛЬКО ТЕКСТОВЫЕ ФАЙЛЫ

Используя регулярное выражение, добавил в подготовленную зону (*staged*) исключительно файлы с расширением `.txt`. Повторный вызов проверки статуса подтвердил, что файл `readme.txt` успешно перешел в индекс, а остальные файлы остались неотслеживаемыми.

```
[24050088@350-glk-pc-09 git-practice]$ git add *.txt
[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 11 – Добавление в индекс только файлов с расширением `.txt`

ЗАДАНИЕ 12. ДОБАВИТЬ ВСЕ ОСТАЛЬНЫЕ ФАЙЛЫ

Процедурой массового добавления перевёл все оставшиеся файлы проекта в индексированную зону. Повторная проверка показала, что теперь абсолютно все три созданных файла готовы к фиксации и находятся в состоянии *staged*.

```
[24050088@350-glk-pc-09 git-practice]$ git add -A
[24050088@350-glk-pc-09 git-practice]$ git status
Текущая ветка: master

Еще нет коммитов

Изменения, которые будут включены в коммит:
(используйте «git rm --cached <файл>...», чтобы убрать из индекса)
    новый файл:    data.json
    новый файл:    readme.txt
    новый файл:    script.sh

[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 12 – Добавление всех файлов в индекс командой `git add -A`

ЗАДАНИЕ 13. СОЗДАТЬ ПЕРВЫЙ КОММИТ

Зафиксировал снимок состояния файлов, создав первый коммит с понятным и кратким описанием внесенных изменений: `initial commit: added base files`. Система успешно сохранила коммит, присвоив ему уникальный хеш `c2698c1607af8be8fb690cb53942b9de36276436`.

```
[24050088@350-glk-pc-09 git-practice]$ git commit -m "initial commit: added base files"
[master (корневой коммит) c2698c1] initial commit: added base files
3 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 data.json
create mode 100644 readme.txt
create mode 100644 script.sh
[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 13 – Создание первого коммита с сообщением

ЗАДАНИЕ 14. ПРОСМОТРЕТЬ ИСТОРИЮ КОММИТОВ

Вывел полный лог разработки для верификации метаданных созданной контрольной точки. Терминал отобразил уникальный полный хеш коммита **c2698c1607af8be8fb690cb53942b9de36276436**, автора **ryabich_1iis_41**, точную дату и время фиксации **Tue Sep 22 16:03:48 2026 +0700**, а также текст комментария.

```
[24050088@350-glk-pc-09 git-practice]$ git log
commit c2698c1607af8be8fb690cb53942b9de36276436 (HEAD -> master)
Author: ryabich_1iis_41 <your.email@example.com>
Date: Tue Sep 22 16:03:48 2026 +0700

    initial commit: added base files
[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 14 – История коммитов с метаданными

ЗАДАНИЕ 15. ВНЕСТИ ИЗМЕНЕНИЯ И ПРОСМОТРЕТЬ РАЗЛИЧИЯ

Открыл текстовый файл `readme.txt` через встроенный консольный редактор Midnight Commander (`mc`) и добавил строку **«6752LOL148840k»**. Далее вызвал команду `git diff` для просмотра построчных различий между рабочей директорией и индексом — утилита наглядно подсветила добавленный фрагмент текста.

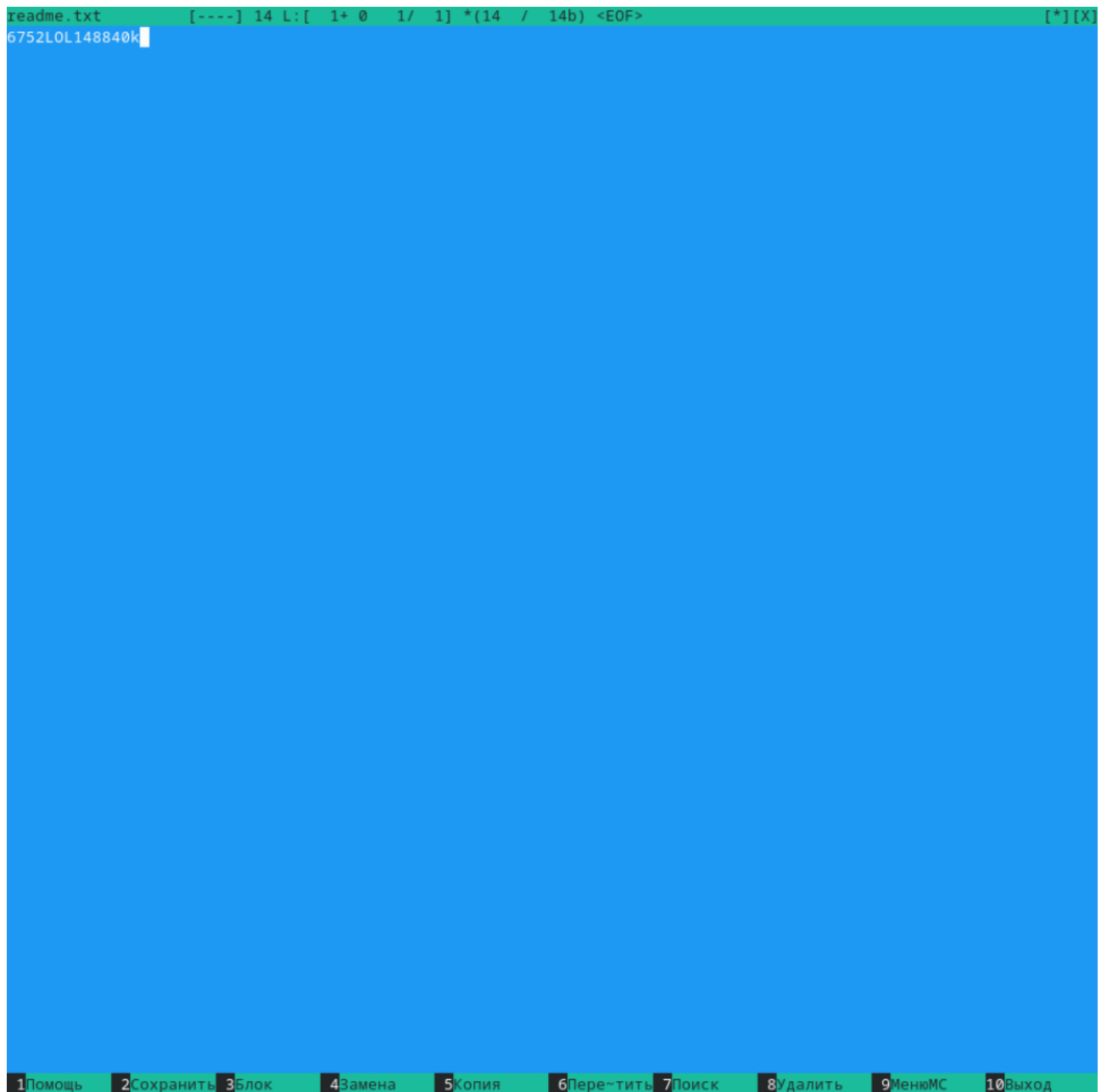


Рисунок 15 – Изменение файла `readme.txt` в МС (строка `6752LOL148840k`)

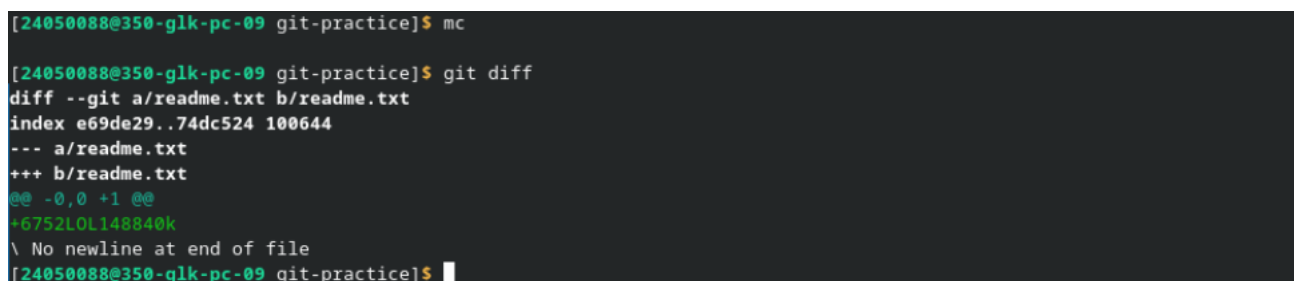


Рисунок 16 – Просмотр различий командой `git diff`

ЗАДАНИЕ 16. СОЗДАТЬ ФАЙЛ `.GITIGNORE`

Для автоматического исключения временных файлов и директорий из процесса отслеживания создал и отредактировал файл `.gitignore`, внеся в него регулярные шаблоны для логов и кэша (`*.log`, `temp/`, `*.tmp`, `.cache/`). Запуск проверки статуса

показал, что сам файл .gitignore корректно распознается системой как неотслеживаемый.

```
[24050088@350-glк-рс-09 git-practice]$ touch .gitignore
[24050088@350-glк-рс-09 git-practice]$ mc
[24050088@350-glк-рс-09 git-practice]$
```

Рисунок 17 – Создание файла .gitignore

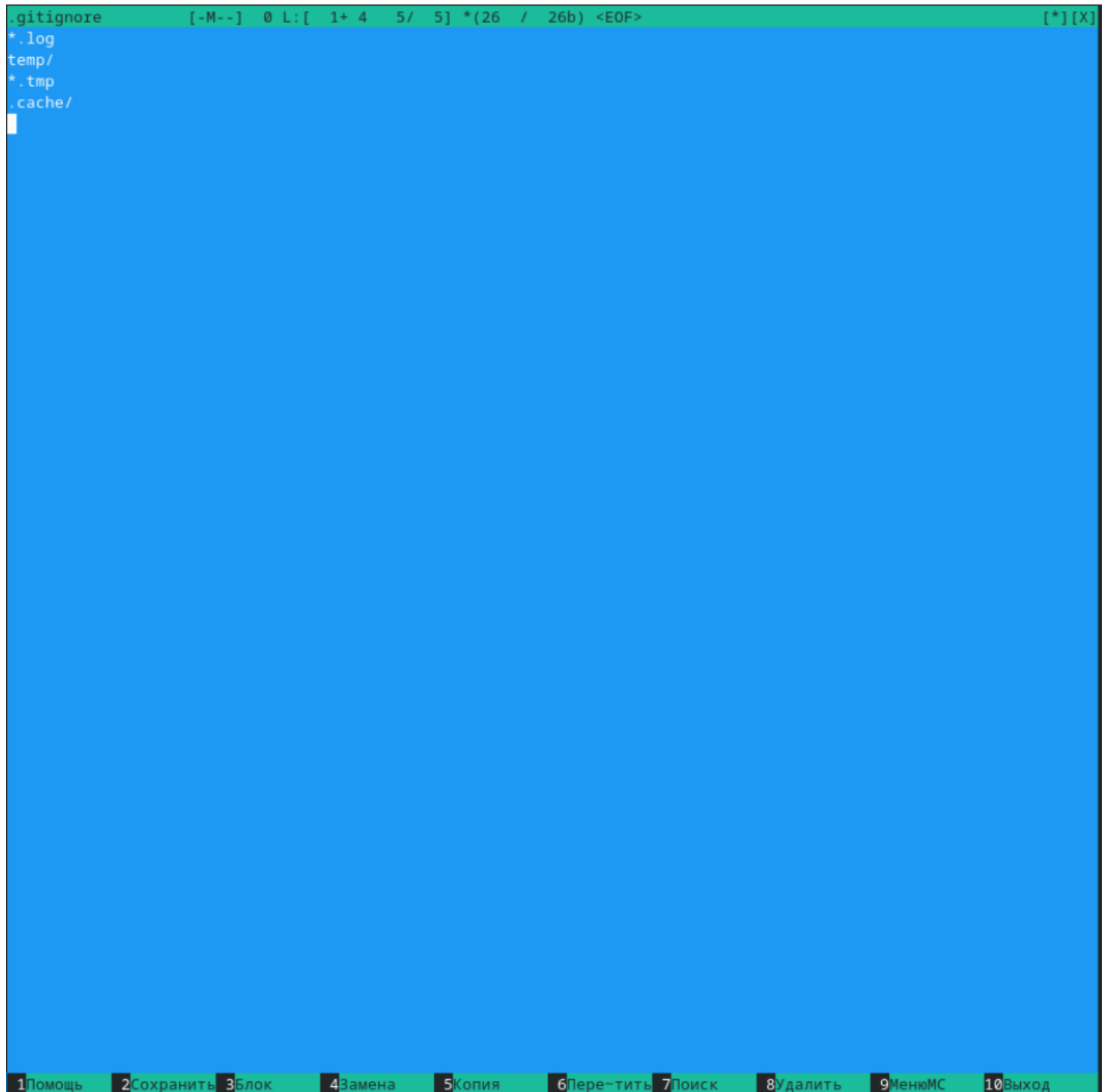


Рисунок 18 – Изменение файла .gitignore в MC

ЗАДАНИЕ 17. СОЗДАТЬ НОВУЮ ВЕТКУ

Сформировал новую изолированную ветку разработки feature/update-readme на основе текущего состояния проекта и автоматически переключился на неё. Проверил список локальных веток и убедился, что активный указатель успешно переместился.

```
[24050088@350-glк-рс-09 git-practice]$ git checkout -b feature/update-readme
Переключились на новую ветку «feature/update-readme»
[24050088@350-glк-рс-09 git-practice]$ git branch -a
* feature/update-readme
  master
[24050088@350-glк-рс-09 git-practice]$
```

Рисунок 19 – Создание ветки *feature/update-readme*, переключение на неё и проверка выбранной ветки

ЗАДАНИЕ 18. ВНЕСТИ ИЗМЕНЕНИЯ, ЗАКОММИТИТЬ И ВЫПОЛНИТЬ СЛИЯНИЕ

В рамках новой ветки через встроенный редактор `mcedit` дополнил файл `readme.txt` новой текстовой строкой «**уеуеуеуеуеуеуе**». После этого зафиксировал изменения новым коммитом с комментарием. Далее вернулся в основную ветку разработки `master` и выполнил процедуру слияния изменений (`git merge`). Слияние успешно завершилось в автоматическом режиме по схеме *Fast-forward* без возникновения конфликтов.

Для финального детального анализа и визуализации всей структуры репозитория вывел историю коммитов в виде графического текстового графа (`git log --oneline --graph --all`), где зафиксировал последовательную цепочку контрольных точек, на последнюю из которых теперь совместно указывают обе ветки.

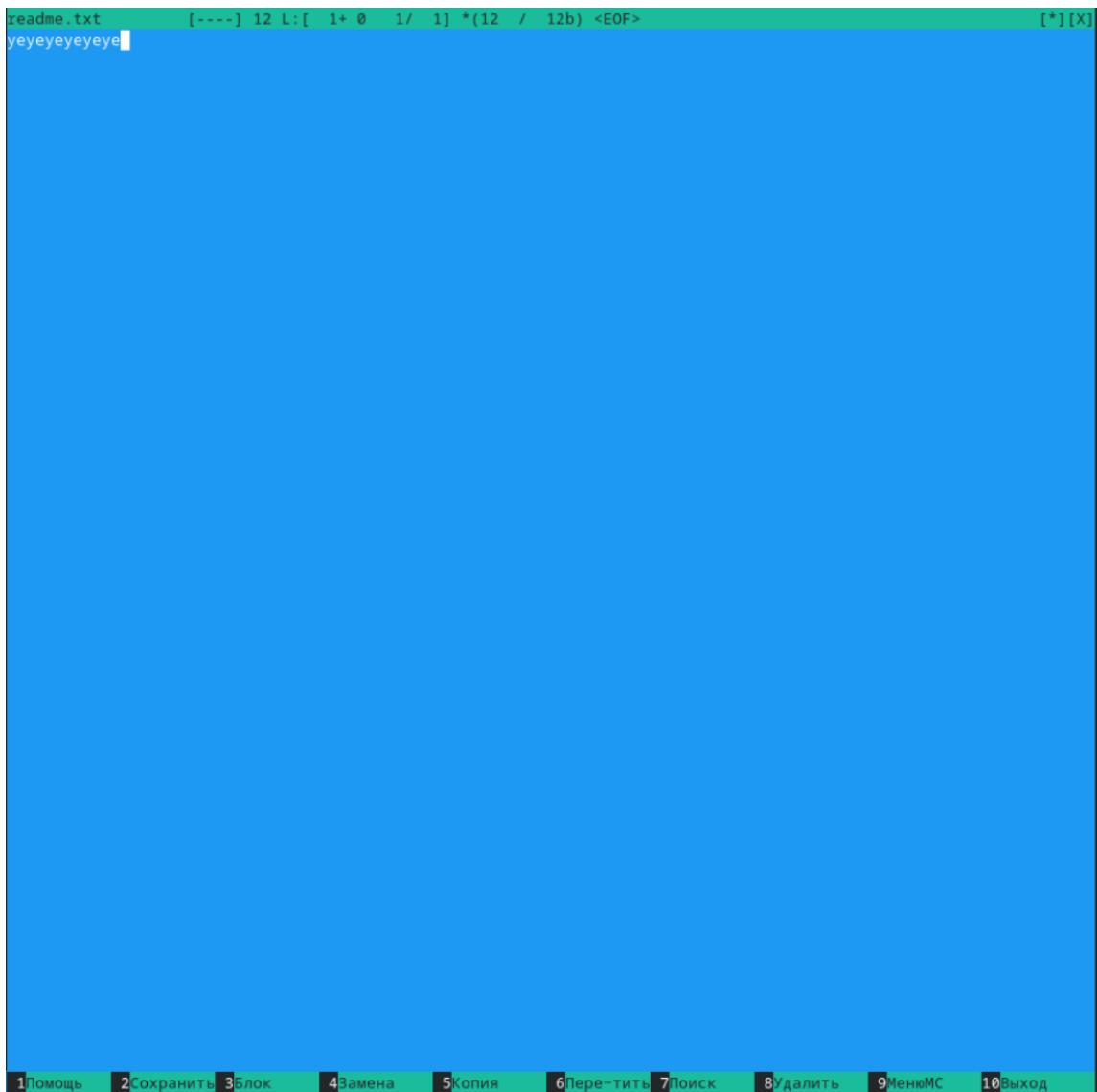


Рисунок 20 – Изменение файла `readme.txt` в МС (строка `yeyeyeyeyeye`)

```
[24050088@350-glk-pc-09 git-practice]$ git add .
[24050088@350-glk-pc-09 git-practice]$ git commit -m "redact readme.txt"
[feature/update-readme 9ed7518] redact readme.txt
2 files changed, 5 insertions(+)
create mode 100644 .gitignore
[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 21 – Создание коммита в новой ветке с комментарием

```
[24050088@350-glk-pc-09 git-practice]$ git checkout master
Переключились на ветку «master»
[24050088@350-glk-pc-09 git-practice]$ git merge feature/update-readme
Обновление c2698c1..9ed7518
Fast-forward
 .gitignore | 4 ++++
 readme.txt | 1 +
 2 files changed, 5 insertions(+)
create mode 100644 .gitignore
[24050088@350-glk-pc-09 git-practice]$ git log --oneline --graph --all
* 9ed7518 (HEAD -> master, feature/update-readme) redact readme.txt
* c2698c1 initial commit: added base files
[24050088@350-glk-pc-09 git-practice]$
```

Рисунок 22 – Переключение на ветку «`master`», объединение двух веток, итоговая история коммитов после слияния веток

Вывод

В ходе лабораторной работы я проверил версию предустановленного пакета Git в РЕД ОС 8.0.3, выполнил глобальную настройку имени пользователя **ryabich_1iis_41** и почты **your.email@example.com**, после чего инициализировал локальный репозиторий в рабочей директории проекта. Все операции я выполнял в консольном терминале с помощью базового набора команд `git init`, `git status`, `git add` и `git commit`, а редактирование файла `readme.txt` осуществлял через встроенный текстовый редактор Midnight Commander (`mc`), последовательно добавляя строки **6752LOL148840k** и **уеуеуеуеуеуеуе** в разных ветках разработки. Построчный анализ изменений я проводил утилитой `git diff`, а слияние изолированной ветки `feature/update-readme` с основной веткой `master` реализовал в автоматическом режиме по схеме *Fast-forward*. В результате я получил полностью настроенный локальный репозиторий со скрытой папкой `.git`, сформировал последовательную историю фиксации данных с уникальным хешем коммита **c2698c1**, обеспечил игнорирование временных файлов через `.gitignore` и успешно объединил ветки с выводом финальной структуры проекта в виде наглядного графа коммитов.

Ответы на контрольные вопросы:

1. Что такое Git и для чего он используется?
 - Распределённая система контроля версий для отслеживания изменений в исходном коде, отката к предыдущим состояниям проекта и организации совместной работы разработчиков.
2. В чём разница между локальным и удалённым репозиторием?
 - Локальный хранится непосредственно на компьютере пользователя в скрытой папке проекта, а удалённый располагается на сервере для синхронизации изменений между членами команды.
3. Какая команда проверяет наличие Git в системе и его версию?
 - `git --version`.
4. Какие команды используются для установки Git в Ubuntu, Fedora и Arch Linux?
 - Для Ubuntu: `sudo apt-get install git`, для Fedora: `sudo dnf install git`, для Arch Linux: `sudo pacman -S git`.
5. Для чего нужна настройка `user.name` и `user.email` в Git?
 - Для обязательной идентификации автора изменений эти метаданные автоматически добавляются в каждое поле `Author` при создании коммитов.
6. В чём разница между командами `git config --global` и `git config` без флага?
 - Флаг `--global` применяет настройки ко всем репозиториям данного пользователя на компьютере, а команда без флага настраивает параметры только для текущего активного репозитория.

7. Что создаётся в директории при выполнении команды `git init`?
 - Скрытая системная папка `.git`, содержащая внутреннюю базу данных репозитория, конфигурационные файлы и всю историю изменений.
8. Для чего предназначена папка `.git` и можно ли её удалять?
 - Она хранит всю историю версионного контроля проекта; удалять её категорически нельзя, иначе проект превратится в обычный каталог, и история коммитов сотрется.
9. Какие четыре состояния файлов существуют в Git?
 - Неотслеживаемое (`untracked`), изменённое (`modified`), подготовленное к фиксации (`staged`) и закоммиченное (`committed`).
10. В чём разница между состояниями `untracked` и `modified`?
 - `Untracked` — это абсолютно новый созданный файл, о котором Git еще не знает, а `modified` — это файл, уже находящийся под контролем версий, но в который были внесены свежие правки.
11. Чем отличается команда `git add .` от `git add -A` и `git add <имя файла>`?
 - `<имя файла>` добавляет одну конкретную правку, индексирует новые и измененные файлы из текущей папки и её подкаталогов, а `-A` добавляет абсолютно все изменения по всему проекту, включая удаленные объекты.
12. Что такое коммит и какие метаданные он содержит?
 - Это основной объект системы контроля версий, представляющий собой снимок состояния файлов; содержит уникальный хэш-идентификатор, имя и почту автора, дату создания и комментарий.
13. Для чего нужна команда `git status` и какую информацию она отображает?
 - Для контроля текущего состояния репозитория; она выводит имя активной ветки, списки неотслеживаемых, измененных и подготовленных к фиксации файлов.
14. Что показывает команда `git diff` и в каких случаях она применяется?
 - Показывает построчные текстовые различия между файлами в рабочей директории и индексом; применяется перед коммитом для проверки внесенных правок.
15. Для чего используется файл `.gitignore` и как формируются его шаблоны?
 - Для автоматического исключения временных файлов, логов или кэша из отслеживания; шаблоны формируются построчно в корневом файле с использованием масок расширений и папок.
16. Что такое ветка в Git и почему её называют подвижным указателем?
 - Это указатель на определенный коммит проекта; она называется подвижной, так как при создании каждой новой фиксации в этой ветке указатель автоматически сдвигается на самый свежий коммит.
17. В чём разница между командами `git checkout master` и `git checkout -b new-branch`?
 - Первая просто переключает рабочую среду на уже существующую ветку `master`, а вторая — создает новую ветку и сразу автоматически переводит пользователя на нее.

18. Что происходит при выполнении команды `git merge` и в каком случае возникает конфликт?

- Происходит слияние изменений из одной ветки в текущую; конфликт возникает, если в объединяемых ветках были параллельно изменены одни и те же строки в одном и том же файле.

19. Как разрешить конфликт слияния и какие маркеры при этом используются в файле?

- Конфликт разрешается вручную в текстовом редакторе путем удаления маркеров <<<<<<< HEAD (текущая ветка), ===== (разделитель), >>>>>>> (сливаемая ветка) с выбором итогового кода, после чего делаются команды `git add` и `git commit`.

20. Для чего применяется команда `git log` и что означает флаг `--oneline --graph --all`?

- Применяется для просмотра истории разработки; `--oneline` сжимает коммиты в одну строку, `--graph` рисует текстовую схему ветвления, а `--all` отображает историю абсолютно всех веток репозитория.